REPUBLIQUE DU SENEGAL



‘

Un Peuple – Un But – Une Foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR, DE LA RECHERCHE ET DE L’INNOVATION

ECOLE POLYTECHNIQUE DE THIÈS



DÉPARTEMENT INFORMATIQUE ET TÉLÉCOMMUNICATIONS

Master 1 en ingénierie logicielle et intelligence artificielle

Cahier de spécifications techniques:

Système de Management et de Suivi des Matériels et Équipements

Compagnie Sahélienne d'Entreprises (CSE)

Master 1 MILIA

Présenté par :

Aliou DRAME

Seydina Mouhamed Lybass POUYE

Michel Sonar NIANE

Encadreur :

Pr. Guissé

Année académique : **2025/2026**

Tables des matières

1. Introduction....4
    1.1 Objet du document....4
    1.2 Rappel du contexte métier....4
    1.3 Portée du document....4
2. Vue d'Ensemble de l'Architecture....5
    2.1 Style architectural retenu....5
    2.2 Découpage en microservices....5
    2.3 Principes directeurs de conception....6
3. Spécification Détaillée des Modules....7
    3.1 Module ms-besoins: Gestion des Expressions de Besoins....7
        Responsabilités....7
        Points d'entrée API principaux....7
    3.2 Module ms-achats: Gestion des Acquisitions et Contrats Fournisseurs....7
        Responsabilités....7
        Points d'entrée API principaux....7
    3.3 Module ms-fournisseurs: Gestion des Fournisseurs et des Prix....7
        Responsabilités....7
        Points d'entrée API principaux....8
    3.4 Module ms-suivi: Suivi de l'Affectation et de l'Utilisation....9
        Responsabilités....9
        Points d'entrée API principaux....9
    3.5 Module ms-reporting: Tableau de Bord Opérationnel et Stratégique....9
        Responsabilités....9
        Indicateurs calculés (rappel synthétique)....9
        Points d'entrée API principaux....9
    3.6 Module ms-chatbot: Assistance Matériel et Logistique....9
        Responsabilités....9
        Points d'entrée API principaux....9
4. Modèle de Données....10
    4.1 Principes de persistance....10
    4.2 Règles de gestion clés....10
    4.3 Renvoi vers l'Annexe UML....10
5. Sécurité....11
    5.1 Authentification et autorisation....11
    5.2 Chiffrement et protection des données....11
    5.3 Traçabilité....11
6. Exigences de Performance et de Qualité....12
    6.1 Performance....12
    6.2 Stratégie de tests....12
    6.3 Maintenabilité....12
7. Stratégie de Déploiement....13
    7.1 Conteneurisation....13

7.2 'Annexe UML..... 13
8. Document Associé.....13

1. Introduction

1.1 Objet du document

Le présent document de spécifications techniques a pour objet de traduire les besoins fonctionnels et non fonctionnels exprimés dans le Cahier des Charges en une architecture logicielle précise, exploitable pour l'implémentation du système en Spring Boot et React.

Il sert de référence technique unique pendant toute la phase de développement.

1.2 Rappel du contexte métier

La Compagnie Sahélienne d'Entreprises (CSE), entreprise de Bâtiment et Travaux Publics fondée en 1970, gère un parc de matériels et d'équipements réparti sur de nombreux chantiers. Le système à développer centralise la gestion des besoins, des achats, des fournisseurs, du suivi opérationnel et des statistiques décisionnelles liées à ce parc.

Pour une description complète des acteurs et des besoins métier, se référer au Cahier des Charges. Ce document de spécifications techniques n'y revient que de manière synthétique, dans la mesure nécessaire à la justification des choix techniques.

1.3 Portée du document

Ce document couvre :

- L'architecture logicielle cible (style architectural, découpage en microservices).
- La description détaillée de chaque module fonctionnel et de ses responsabilités techniques.
- Les contrats d'interface (API REST) entre les composants du système.
- Le modèle de données et les règles de gestion associées.
- La stratégie de sécurité, de performance et de déploiement.
- Les renvois vers l'annexe UML pour toute représentation graphique.

Ce document ne couvre pas le code source lui-même ni le détail des diagrammes UML (objet de l'Annexe UML).

2. Vue d'Ensemble de l'Architecture

2.1 Style architectural retenu

Le système adopte une architecture orientée microservices, où chaque module fonctionnel majeur du Cahier des Charges est implémenté comme un service Spring Boot indépendant, déployable séparément. Cette architecture est complétée par le patron Modèle-Vue-Contrôleur (MVC) au sein de chaque service pour structurer la couche applicative.

Les échanges entre services suivent deux modes de communication complémentaires :

- Communication synchrone (REST/JSON) pour les opérations nécessitant une réponse immédiate (consultation, validation).
- Communication asynchrone pour la diffusion d'événements métier (notifications, mise à jour de tableaux de bord) sans coupler directement les services émetteurs et récepteurs.

2.2 Découpage en microservices

Le système est décomposé en six microservices métier, conformes aux modules fonctionnels du Cahier des Charges, complétés par deux services transverses (passerelle et authentification). Le détail visuel de leurs interactions figure dans l'Annexe UML, section « Diagramme de Composants ».

Microservice	Module fonctionnel correspondant	Responsabilité principale
ms-besoins	Gestion des expressions de besoins des chantiers	Saisie, workflow de validation, consolidation des demandes
ms-achats	Gestion des acquisitions et contrats fournisseurs	Appels d'offres, réception des offres, contrats
ms-fournisseurs	Gestion des fournisseurs et des prix	Référentiel fournisseurs, évaluation, catalogue de prix
ms-suivi	Suivi de l'affectation et de l'utilisation des matériels	Affectations, mouvements, maintenance préventive/corrective
ms-reporting	Tableau de bord opérationnel et stratégique	KPIs, génération de rapports, exports
ms-chatbot	Chatbot d'assistance matériel et logistique	Recherche en langage naturel, interrogation des données
ms-gateway	Transverse	Point d'entrée unique, routage, sécurité, rate limiting
ms-auth	Transverse	Authentification, gestion des rôles (intégration Keycloak)

Pour la cartographie complète des flux entre ces microservices (qui appelle qui, par quel protocole), se référer à l'Annexe UML, Diagramme de Composants.

2.3 Principes directeurs de conception

- Autonomie des données : chaque microservice métier possède son propre schéma de base de données ; aucun accès direct cross-service à une base étrangère.
- API-first : tout échange inter-service passe par un contrat REST documenté (Swager), même en environnement de développement.
- Dégradation progressive : les fonctionnalités critiques (besoins, achats, suivi) restent opérationnelles même si un service secondaire (chatbot, reporting) est indisponible.
- Sécurité centralisée : aucune logique d'authentification n'est dupliquée dans les microservices métier ; elle est déléguée à ms-auth et à la passerelle.

3. Spécification Détaillée des Modules

Cette section détaille, pour chaque service métier, ses responsabilités techniques, son modèle de données simplifié et ses principaux points d'entrée API. La structure complète des classes est disponible dans l'Annexe UML, Diagramme de Classes.

3.1 Module ms-besoins: Gestion des Expressions de Besoins

Responsabilités

- Recevoir et valider la saisie d'une expression de besoin (chantier, désignation, quantité, spécifications, priorité).
- Orchestrer le workflow de validation multi-niveaux (Chef de Chantier , Responsable Matériel).
- Déterminer automatiquement si le besoin peut être satisfait par affectation directe (matériel disponible) ou doit déclencher un achat.
- Exposer l'historique complet et les statuts de chaque demande.

Points d'entrée API principaux

- POST /api/besoins: Soumettre une expression de besoin
- GET /api/besoins/{id}: Consulter le statut d'une demande
- PATCH /api/besoins/{id}/valider: Valider un besoin (rôle ResponsableMateriel)
- PATCH /api/besoins/{id}/rejeter : Rejeter un besoin avec motif obligatoire
- GET /api/besoins?chantierId=&statut=: Lister les besoins filtrés

3.2 Module ms-achats: Gestion des Acquisitions et Contrats Fournisseurs

Responsabilités

- Générer un appel d'offres à partir d'un besoin validé nécessitant un achat.
- Réceptionner les offres déposées par les fournisseurs jusqu'à la clôture.
- Permettre la comparaison multicritère des offres (prix, délai, qualité technique).
- Générer le contrat d'achat à partir de l'offre retenue et le transmettre pour validation budgétaire.

Points d'entrée API principaux

- POST /api/appels-offres: Publier un appel d'offres
- GET /api/appels-offres/{id}/offres: Lister les offres reçues (anonymisées avant clôture)
- POST /api/appels-offres/{id}/attribuer: Attribuer le marché à une offre
- POST /api/contrats: Générer un contrat d'achat depuis une offre retenue
- GET /api/contrats/{id}: Consulter un contrat et son statut

3.3 Module ms-fournisseurs: Gestion des Fournisseurs et des Prix

Responsabilités

- Maintenir le référentiel des fournisseurs (informations légales, spécialités, statut).

- Calculer et mettre à jour le score de performance après chaque marché exécuté.
- Exposer le catalogue de prix par fournisseur et par catégorie de matériel.
- Gérer la suspension ou le blacklistage d'un fournisseur.

Points d'entrée API principaux

- GET /api/fournisseurs?specialite=&statut=:Lister les fournisseurs filtrés
- POST /api/fournisseurs/{id}/evaluer: Enregistrer une évaluation post-marché
- PATCH /api/fournisseurs/{id}/suspendre: Suspendre un fournisseur
- PATCH /api/fournisseurs/{id}/blacklister: Blacklister (validation Direction Générale requise)

3.4 Module ms-suivi: Suivi de l'Affectation et de l'Utilisation

Responsabilités

- Gérer le cycle de vie complet d'un matériel (DISPONIBLE, AFFECTE, EN_MAINTENANCE).
- Planifier et tracer les affectations de matériel aux chantiers, avec détection de conflits.
- Notifier le Chef de Chantier à la réception et déclencher les alertes de panne.

Points d'entrée API principaux

- POST /api/materiels/{id}/affecter: Affecter un matériel à un chantier
- POST /api/materiels/{id}/confirmer-reception: Confirmer la réception sur chantier
- POST /api/maintenances: Déclarer une intervention de maintenance
- GET /api/materiels?etat=&categorie=: Lister les matériels filtrés

3.5 Module ms-reporting: Tableau de Bord Opérationnel et Stratégique

Responsabilités

- Agréger les données issues des autres microservices pour produire les KPIs.
- Fournir des vues différenciées selon le profil de l'acteur connecté (Direction Générale, Responsable Matériel, Chef de Chantier, DAF, Responsable Achats).
- Générer des exports PDF et Excel paramétrables.

Indicateurs calculés (rappel synthétique)

- Taux d'utilisation des matériels par type, chantier et période.
- Disponibilité globale et par catégorie de matériel.
- Performance des fournisseurs (classement par score).
- Indicateurs financiers : montants engagés vs réalisés.

Points d'entrée API principaux

- GET /api/reporting/kpis?profil=&periode=: Obtenir les KPIs selon le profil de l'utilisateur
- GET /api/reporting/export?format=pdf|xlsx: Générer un export

3.6 Module ms-chatbot: Assistance Matériel et Logistique

Responsabilités

- Recevoir une requête en langage naturel et la transmettre au moteur NLP (LLM externe).
- Interroger les microservices ms-suivi et ms-besoins pour contextualiser la réponse avec des données réelles et à jour.
- Conserver un historique de conversation consultable par l'utilisateur.

Points d'entrée API principaux

- POST /api/chatbot/query: Envoyer une question en langage naturel
- GET /api/chatbot/historique: Consulter l'historique des conversations de l'utilisateur

4. Modèle de Données

Le modèle de données complet, avec attributs typés, méthodes et relations, est représenté graphiquement dans l'Annexe UML, Diagramme de Classes. Cette section en rappelle uniquement les règles de gestion essentielles pour la conception des schémas PostgreSQL.

4.1 Principes de persistance

- Chaque microservice métier dispose de son propre schéma PostgreSQL, isolé des autres (pattern « Database per Service »).
- Les identifiants primaires sont des UUID générés côté application, pour éviter toute collision lors de migrations ou de fusions de données entre environnements.
- Les énumérations (statuts, catégories, priorités) sont implémentées comme des types ENUM PostgreSQL natifs, garantissant l'intégrité des valeurs au niveau base de données, en complément de la validation applicative.

4.2 Règles de gestion clés

Règle	Description
01	Un BesoinMateriel ne peut être validé que par un utilisateur ayant le rôle ResponsableMateriel.
02	Un AppelOffre ne peut recevoir de nouvelle Offre après sa date limite de réponse (dateLimite).
03	Une Offre ne peut devenir un ContratAchat que si le statut de l'AppelOffre associé est ATTRIBUE.
04	Un Materiel ne peut être affecté à un nouveau chantier que si son état est DISPONIBLE.
06	Un ContratAchat ne peut être exécuté (déclenchement de livraison) qu'après validation de l'engagement budgétaire par la DAF.
07	Le score de performance d'un Fournisseur est recalculé automatiquement après chaque clôture de réception de Materiel le concernant.

4.3 Annexe UML

Les classes Utilisateur (et ses s sous-classes par héritage), Chantier, BesoinMateriel, AppelOffre, Offre, ContratAchat, Materiel, ainsi que toutes leurs relations et cardinalités, sont détaillées dans l'Annexe UML, Section « Diagramme de Classes ».

5. Sécurité

5.1 Authentification et autorisation

L'authentification est entièrement déléguée à Keycloak (protocole OAuth2/OIDC). Aucun microservice métier ne stocke ni ne vérifie de mot de passe directement.

- Le client (application React) obtient un token JWT auprès de Keycloak après authentification.
- Chaque requête vers l'API Gateway porte ce token dans l'en-tête Authorization.
- La passerelle (ms-gateway) valide le token avant de router la requête vers le microservice cible.
- L'autorisation fine (RBAC) est appliquée au niveau de chaque microservice via les rôles encodés dans le token (claims), correspondant aux sept acteurs définies dans le modèle de classes.

5.2 Chiffrement et protection des données

- Toutes les communications transitent en HTTPS (TLS), y compris en interne entre microservices.
- Les données sensibles au repos (montants de contrats, coordonnées bancaires fournisseurs) sont chiffrées en base via AES-256.

5.3 Traçabilité

Toute action de création, modification ou suppression sur une entité métier (BesoinMateriel, ContratAchat, Materiel) génère un événement d'audit horodaté et consommé par un service de journalisation centralisé. Chaque entrée d'audit enregistre : l'identifiant de l'utilisateur, l'action effectuée, l'entité concernée et l'horodatage.

6. Exigences de Performance et de Qualité

6.1 Performance

Critère	Cible
Temps de réponse API	< 2 secondes pour 95 % des requêtes sous charge normale (200 utilisateurs simultanés)
Pagination	Obligatoire pour toute liste retournant plus de 50 éléments
Disponibilité	≥ 99 % en heures ouvrables (6h–22h, 7 jours sur 7)

6.2 Stratégie de tests

- Couverture de tests unitaires cible : 80 % minimum sur la logique métier de chaque microservice (services et règles de gestion).
- Tests d'intégration systématiques sur les contrats d'API REST, avant toute mise en commun des microservices développés en parallèle par les membres de l'équipe.
- Tests de bout en bout sur les trois scénarios critiques : soumission/validation d'un besoin, cycle complet d'achat, confirmation de réception et clôture de maintenance.

6.3 Maintenabilité

- Code versionné sur Git, avec une branche par microservice et des revues de code croisées entre les trois membres avant fusion sur la branche principale.
- Documentation API générée automatiquement via Swagger, exposée par chaque microservice .
- Respect des conventions de nommage Java standard (camelCase pour les méthodes et variables, PascalCase pour les classes) pour une cohérence avec le modèle de classes de l'Annexe UML.

7. Stratégie de Déploiement

7.1 Conteneurisation

L'ensemble des composants est packagé sous forme de conteneurs Docker, orchestrés par Docker Compose pour la durée du projet académique. Cette approche est volontairement simplifiée par rapport à un orchestrateur de production (Kubernetes), compte tenu du périmètre et de la durée du projet.

7.2 'Annexe UML

La cartographie physique complète (postes clients, serveurs, ports réseau, protocoles de communication entre chaque conteneur) est représentée dans l'Annexe UML, Section « Diagramme de Déploiement ». Ce document de spécifications techniques n'en retient que les environnements et leur usage, déjà décrits ci-dessus.

8. Document Associé

les diagrammes UML (diagramme de classes, diagramme de composants, diagramme de déploiement) ne sont PAS reproduits graphiquement dans ce document. Ils font l'objet d'un document séparé intitulé « Annexe UML, Diagrammes de Modélisation », auquel ce document renvoie explicitement à chaque section concernée.